

<HLS Programming with FPGAs>

The HLS Book

Chapter 7

Dense Matrix Multiplication 1

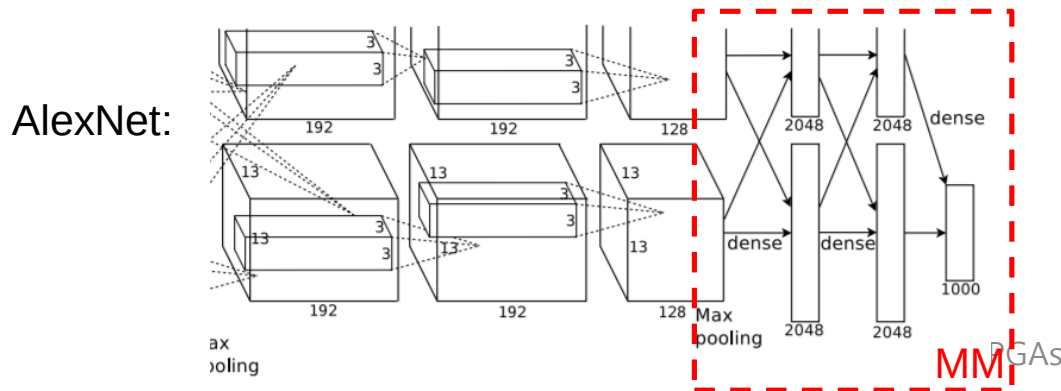
Choi, Young-kyu

Learning Objectives

- Implement dense matrix multiplication with HLS
- Generate a testbench for matrix multiplication and quantify its performance
- Understand the difficulty in multiplying large matrices

7.1 Background

- Fundamental operation in numerical algorithms
- Provides a way to compose linear transforms
- Applications
 - Linearly changing coordinates (e.g. translation and rotation in graphics)
 - High dimensional problems in statistical physics
 - Graph operations (e.g. all-pairs shortest path problem)
 - Fully-connected layer in CNN/DNN inference



Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "Imagenet classification with deep convolutional neural networks." *Advances in neural information processing systems* 25 (2012): 1097-1105.

- Matrix product:

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1m} \\ A_{21} & A_{22} & \cdots & A_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \cdots & A_{nm} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} B_{11} & B_{12} & \cdots & B_{1p} \\ B_{21} & B_{22} & \cdots & B_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ B_{m1} & B_{m2} & \cdots & B_{mp} \end{bmatrix}$$

$$\mathbf{AB} = \begin{bmatrix} (\mathbf{AB})_{11} & (\mathbf{AB})_{12} & \cdots & (\mathbf{AB})_{1p} \\ (\mathbf{AB})_{21} & (\mathbf{AB})_{22} & \cdots & (\mathbf{AB})_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ (\mathbf{AB})_{n1} & (\mathbf{AB})_{n2} & \cdots & (\mathbf{AB})_{np} \end{bmatrix} \quad (\mathbf{AB})_{ij} = \sum_{k=1}^m A_{ik} B_{kj}.$$

- Example: $\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \\ B_{31} & B_{32} \end{bmatrix}$

$$\mathbf{AB} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} + A_{13}B_{31} & A_{11}B_{12} + A_{12}B_{22} + A_{13}B_{32} \\ A_{21}B_{11} + A_{22}B_{21} + A_{23}B_{31} & A_{21}B_{12} + A_{22}B_{22} + A_{23}B_{32} \end{bmatrix}$$

Code for Matrix Multiplication

- C Code:

```
void matrixmul(int A[N][M], int B[M][P], int AB[N][P]) {  
    /* for each row and column of AB */  
    row: for(int i = 0; i < N; ++i) {  
        col: for(int j = 0; j < P; ++j) {  
            /* compute (AB)i,j */  
            int ABij = 0;  
            product: for(int k = 0; k < M; ++k) {  
                ABij += A[i][k] * B[k][j];  
            }  
            AB[i][j] = ABij;  
        }  
    }  
}
```

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1m} \\ A_{21} & A_{22} & \cdots & A_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \cdots & A_{nm} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} B_{11} & B_{12} & \cdots & B_{1p} \\ B_{21} & B_{22} & \cdots & B_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ B_{m1} & B_{m2} & \cdots & B_{mp} \end{bmatrix}$$

$$\mathbf{AB} = \begin{bmatrix} (\mathbf{AB})_{11} & (\mathbf{AB})_{12} & \cdots & (\mathbf{AB})_{1p} \\ (\mathbf{AB})_{21} & (\mathbf{AB})_{22} & \cdots & (\mathbf{AB})_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ (\mathbf{AB})_{n1} & (\mathbf{AB})_{n2} & \cdots & (\mathbf{AB})_{np} \end{bmatrix}$$

- Complexity: $O(N \cdot P \cdot M)$

Pipelining & Unrolling Optimization

- Choosing a loop to pipeline:

```
void matrixmul(int A[N][M], int B[M][P], int AB[N][P]) {  
    /* for each row and column of AB */  
    row: for(int i = 0; i < N; ++i) {  
        col: for(int j = 0; j < P; ++j) {  
            /* compute (AB)i,j */  
            int ABij = 0;  
            product: for(int k = 0; k < M; ++k) {  
                ABij += A[i][k] * B[k][j];  
            }  
            AB[i][j] = ABij;  
        }  
    }  
}
```

Loop I

Loop J

Loop K

Which one?

#pragma HLS pipeline

- $P \times M$ MAC operations per cycle
- Unrolls loop j and loop k
- Needs to fully partition B, partition A by M, AB by P

- M MAC operations per cycle
- Unrolls loop k
- Needs to partition A and B by M
- One MAC operation per cycle
- No unrolling
- No need to partition A and B

*MAC : multiply-accumulate

Pipelining & Unrolling Optimization

- Choosing a loop to pipeline:

Highest throughput 😊
Most resource consumption ☹️

```
void m... (P) {  
    /* for each row and column of AB */  
    row: for(int i = 0; i < N; ++i) {  
        col: for(int j = 0; j < P; ++j) {  
            /* compute (AB)i,j */  
            int ABij = 0;  
            product: for(int k = 0; k < M; ++k) {  
                ABij += A[i][k] * B[k][j];  
            }  
            AB[i][j] = ABij;  
        }  
    }  
}
```

Lowest throughput ☹️
Least resource consumption 😊

- $P \times M$ MAC operations per cycle
- Unrolls loop j and loop k
- Needs to fully partition B, partition A by M, AB by P

Which one?

#pragma HLS pipeline

- M MAC operations per cycle
- Unrolls loop k
- Needs to partition A and B by M
- One MAC operation per cycle
- No unrolling
- No need to partition A and B

- Pipelining loop J:

```
void matrixmul(int A[N][M], int B[M][P], int AB[N][P]) {
    #pragma HLS ARRAY_PARTITION variable=A complete dim=2
    #pragma HLS ARRAY_PARTITION variable=B complete dim=1
    /* for each row and column of AB */
    row: for(int i = 0; i < N; ++i) {
        col: for(int j = 0; j < P; ++j) { //pipelined
            #pragma HLS PIPELINE II=1
            /* compute (AB)i,j */
            int ABij = 0;
            product: for(int k = 0; k < M; ++k) { //unrolled
                ABij += A[i][k] * B[k][j];
            }
            AB[i][j] = ABij;
        }
    }
}
```

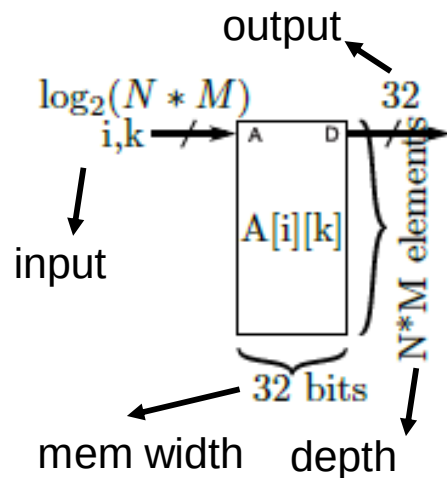

Vitis HLS Pragma : **ARRAY_PARTITION**

- Syntax: **#pragma HLS ARRAY_PARTITION**
variable=<name> <type> factor=<int> dim=<int>

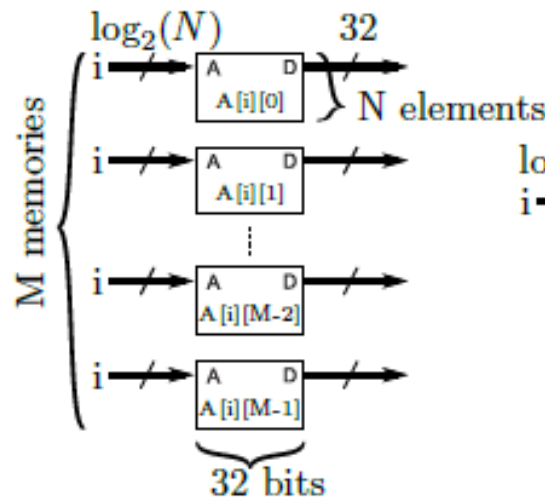
- (type=)block : 
- (type=)cyclic : 
- (type=)complete : completely partitions the array
- factor= the number of partitioned arrays
 - factor is not needed for completely partitioned array
- dim= specifies which dimension of multi-dimension array is partitioned
- Ex)

```
void matrixmul(int A[N][M], int B[M][P], int AB[N][P]) {  
    #pragma HLS ARRAY_PARTITION: variable=A complete dim=2  
    #pragma HLS ARRAY_PARTITION: variable=B complete dim=1
```

- Memory partition types

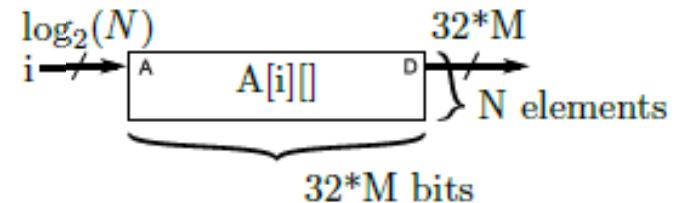


<No Partitioning>



<ARRAY_PARTITION>

- Pro: High memory bandwidth
- Con: Many different control logics and memories (possible underutilization)



<ARRAY_RESHAPE>

- Pro: All partitions will be merged to a single (wide) physical memory → **high BW & simple control logic**
- Con: All elements must have the same address

- Using reshaped memory

```

void matrixmul(int A[N][M], int B[M][P], int AB[N][P]) {
    #pragma HLS ARRAY_RESHAPE variable=A complete dim=2
    #pragma HLS ARRAY_RESHAPE variable=B complete dim=1
    /* for each row and column of AB */
    row: for(int i = 0; i < N; ++i) {
        col: for(int j = 0; j < P; ++j) {
            #pragma HLS PIPELINE II=1
            /* compute (AB)i,j */
            int ABij = 0;
            (Unrolled loop) product: for(int k = 0; k < M; ++k) {
                ABij += A[i][k] * B[k][j];
            }
            AB[i][j] = ABij;
        }
    }
}

```

<Figure 7.1>

Testbench Generation for MM

- Assumes Vitis HLS environment

```
DTYPE A[SIZE][SIZE], B[SIZE][SIZE];
DTYPE AB_sw[SIZE][SIZE], AB_hw[SIZE][SIZE];

init_matrices: for(int i = 0; i < SIZE; i++){
    for(int j = 0; j < SIZE; j++){
        A[i][j] = rand() % 8;
        B[i][j] = rand() % 8;
        AB_sw[i][j] = 0;
        AB_hw[i][j] = 0;
    }
}
```

<Data Initialization>

<Kernel launch>

```
mm(A, B, AB_hw, SIZE);
```

<Golden output generation>

```
mm_sw(A, B, AB_sw);
```

```
void mm_sw(DTYPE A[SIZE][SIZE],
           DTYPE B[SIZE][SIZE], DTYPE out[SIZE][SIZE]){
    DTYPE sum = 0;
    for(int i = 0; i < SIZE; i++){
        for(int j = 0; j < SIZE; j++){
            sum = 0;
            for(int k = 0; k < SIZE; k++){
                sum = sum + A[i][k] * B[k][j];
            }
            out[i][j] = sum;
        }
    }
}
```

```
int fail = 0;
for(int i = 0; i < SIZE; i++){
    for(int j = 0; j < SIZE; j++){
        if(AB_sw[i][j] != AB_hw[i][j]) {
            fail=1;
        }
    }
}
```

<Correctness check>

```
if(fail==1){
    printf("FAILED!\n");
}
else{
    printf("PASSED!\n");
}
```

<Performance measurement>

- GFLOPS : Giga FLoating-point Operations Per Second
 - Commonly used performance metric in high-performance computing
 - Variant: GOPS (integers)
 - Ex) (square-shaped) Dense MM:
 - $\text{GFLOPS} = 2 * \text{matrix_size}^3 / \text{time_taken_in_ns}$
 - Constant “2” reflects one addition and one multiplication per iteration (MAC operation)
 - Vitis testbench:

```
double exec_time = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start).count();
double gops = double(SIZE) * SIZE * SIZE * 2 / (exec_time);
std::cout << "Time: " << exec_time*1e-9 << ", GOPS: " << gops << std::endl;
```

Exercise

- Compute GFLOPS in the below FIR code:

```
s_loop: for (int s = 0; s < num_sample; s++) {  
    acc_t acc = 0;  
    i_loop: for (int i = N - 1; i >= 0; i--){  
        shift_reg[i] = (i==0) ? x[s] : shift_reg[i - 1];  
        data_t x_temp = (i==0) ? x[s] : shift_reg[i];  
        acc += x_temp * c[i];  
    }  
    y[s] = acc;  
}
```

- num_sample : 1000
- N = 11
- Data type: float
- Clock frequency: 250MHz
- Execution cycle : 1010 cycles

Solution

- Compute GFLOPS in the below FIR code:

```
s_loop: for (int s = 0; s < num_sample; s++) {  
    acc_t acc = 0;  
    i_loop: for (int i = N - 1; i >= 0; i--){  
        shift_reg[i] = (i==0) ? x[s] : shift_reg[i - 1];  
        data_t x_temp = (i==0) ? x[s] : shift_reg[i];  
        acc += x_temp * c[i];  
    }  
    y[s] = acc;  
}
```

- num_sample : 1000
- N = 11
- Data type: float
- Clock frequency: 250MHz
- Execution cycle : 1010

- 11 additions and 11 multiplications per sample → 22000 floating-point operations
- Execution time = 1010 cycles / 0.250 GHz
- GFLOPS : $22000 * 0.250 / 1010 = 5.4$ GFLOPS

Large Matrix Multiplication

- Until now, we assumed that matrices A , B , AB fit inside the FPGA internal memory
- F1 internal memory size
 - BRAM: 9.5MB
 - URAM: 34MB
- Q: What if the size of matrices exceeds the size of the internal memory?
- Possible solution: For each iteration, read/write one element of $A/B/AB$ from the FPGA device DRAM (no data stored in internal memory)

- DRAM↔FPGA data transfer size comparison
 - Assuming we are multiplying two matrices of size $N \times N$, int type
 - Case 1) If all matrices stored in FPGA internal memory
 - All matrices read/written to DRAM only once
 - DRAM transfer size : $3 * N * N * 4 = 12 * N * N$ (Bytes)
 - Case 2) If NO matrices stored in FPGA internal memory

The diagram shows the following code snippet with annotations:

```

int ABij = 0;
product: for(int k = 0; k < N; ++k)
{
  ABij += A[i][k] * B[k][j];
}
AB[i][j] = ABij;
  
```

Annotations:

- A purple dashed arrow labeled "register" points to the variable `ABij` in the inner loop.
- A green dashed arrow labeled "from DRAM" points to the `B[k][j]` access in the inner loop.
- A blue dashed arrow labeled "to DRAM" points to the `AB[i][j]` assignment at the end of the inner loop.

- DRAM transfer size $\approx 2 * N * N * N * 4 = 8 * N * N * N$ (Bytes)
- New problem: Needs to transfer much larger data from DRAM

- Ex) Bottleneck analysis if no matrices are stored in FPGA internal memory
 - Assume:
 - Multiplying two matrices of size 4096×4096 , int type
 - Performs 1024 MAC operations per cycle
 - Note: 6840 DSPs on F1
 - Transfers 64B per cycle from DRAM
 - Note: 4CHs * 64B/cycle ideal DRAM BW on F1
 - Computation cycles:
 - $4096 * 4096 * 4096 / 1024 = 64\text{M}$ cycles
 - DRAM transfer cycles:
 - $\approx 2 * 4096 * 4096 * 4096 * 4 / 64 = 8096\text{M}$ cycles
- DRAM BW becomes the bottleneck

- Reason for the DRAM bottleneck: **Lack of data reuse**

– Q) How many times can A_{11} be reused?

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1m} \\ A_{21} & A_{22} & \cdots & A_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \cdots & A_{nm} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} B_{11} & B_{12} & \cdots & B_{1p} \\ B_{21} & B_{22} & \cdots & B_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ B_{m1} & B_{m2} & \cdots & B_{mp} \end{bmatrix}$$

$$\mathbf{AB} = \begin{bmatrix} (AB)_{11} & (AB)_{12} & \cdots & (AB)_{1p} \\ (AB)_{21} & (AB)_{22} & \cdots & (AB)_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ (AB)_{n1} & (AB)_{n2} & \cdots & (AB)_{np} \end{bmatrix}$$

→ A_{11} can be reused up to **p** times if stored in FPGA internal memory

Q) How can we exploit this data reuse even when multiplying large matrices?

Summary

- Dense matrix multiplication is composed of 3 nested loops
- We should decide which loop to pipeline and unroll depending on the amount of parallelism and memory partitioning
 - Typically pipelines the J loop, unroll the K loop, and array-reshape A and B arrays
- Calculated GFLOPS to evaluate matrix multiplication performance in the testbench
- DRAM BW becomes the bottleneck if the matrices cannot fit in the FPGA internal memory